AI Agent Platform Engagement

InnovateCorp — Deployed Architect Take-Home

Part 1: LangSmith Platform (self-hosted) architecture

Part 2: Multi-agent travel-booking system, built, evaluated, and run live

Agenda

1. Part 1 — LangSmith Platform architecture (self-hosted)
2. Part 2 — Agent system: design, evaluation, results
3. Business value
4. Friction log & product feedback
5. Q&A

Part 1

LangSmith Platform Architecture

Why self-hosted?

- InnovateCorp needs agent observability/evaluation at production scale
- Data residency / compliance requirements rule out sending trace data to a third-party cloud
- InnovateCorp already runs Kubernetes — reuse that investment
- **Scope note:** this is a conceptual design exercise (per the assignment) — no Terraform, no Helm values, diagrams and reasoning only

Core components — six application services

Service	Responsibility
Frontend	Nginx-served UI + API routing
Backend	CRUD API, business logic, hub API
Platform Backend	Auth, high-volume run ingestion
Queue	Async trace/feedback ingestion, retries
Playground	Forwards requests to LLM providers
ACE Backend	Sandboxed custom code execution

Grounded in LangChain's published self-hosted docs — fetched live, not from memory.

Three stateful stores

Store	Holds
PostgreSQL	Orgs, users, projects, API keys — transactional data
ClickHouse	Trace spans, run metadata, eval results — the OLAP store, by far the largest
Redis / Valkey	Ingestion queue, pub/sub, short-lived cache
Blob storage	Large payloads, attachments

Externalization: the easy calls and the real one

- Postgres → RDS / Cloud SQL — standard OLTP, no reason to self-manage
- Redis → ElastiCache / Memorystore — managed queue durability matters under burst load
- Blob storage → S3 / GCS — trivial call
- **ClickHouse is the real judgment call.** LangChain's own AWS reference architecture runs it in-cluster on EBS and marks it **"dev/POC only, no replication."**
    - Production needs either a managed ClickHouse offering, or a deliberately-operated multi-replica cluster — not "scale it up later"

Deployment topology (AWS/EKS reference)

- VPC: public subnets (NAT, ALB) / private subnets (EKS, RDS, ElastiCache)
- EKS managed node group: **m5.4xlarge** (16 vCPU / 64 GB) minimum
- ClickHouse: **dedicated node**, 4 vCPU / 16 GB, SSD-backed storage (≥7000 IOPS, ≥1000 MiB/s)
- IRSA (not static credentials) for S3 access; External Secrets Operator + SSM for secrets
- Egress allowlisted to `beacon.langchain.com` for license verification (unless air-gapped)

Full diagram in `docs/langsmith-platform-architecture.html`.

Scaling approach

Stateless tier (Frontend, Backend, Platform Backend, Queue, Playground, ACE Backend) → standard HPA + cluster autoscaler. Platform Backend and Queue matter most — they sit in the high-volume ingestion path.

Stateful tier

→ Postgres/Redis scale the conventional managed-service way (read replicas if needed).

→ **ClickHouse scales with agent trace volume directly** — every new production agent is more spans, not more users. Plan for this from day one, not as a retrofit.

Part 1 — open questions (stated honestly)

- Multi-AZ / HA requirements for Postgres and ClickHouse — depends on InnovateCorp's actual SLA
- Expected trace volume and retention window — drives the ClickHouse sizing/externalize decision
- Air-gapped network posture — changes the licensing approach
- SSO/identity provider integration — org-specific

Not guessed at — these need InnovateCorp's own constraints, not an assumption.

Part 2

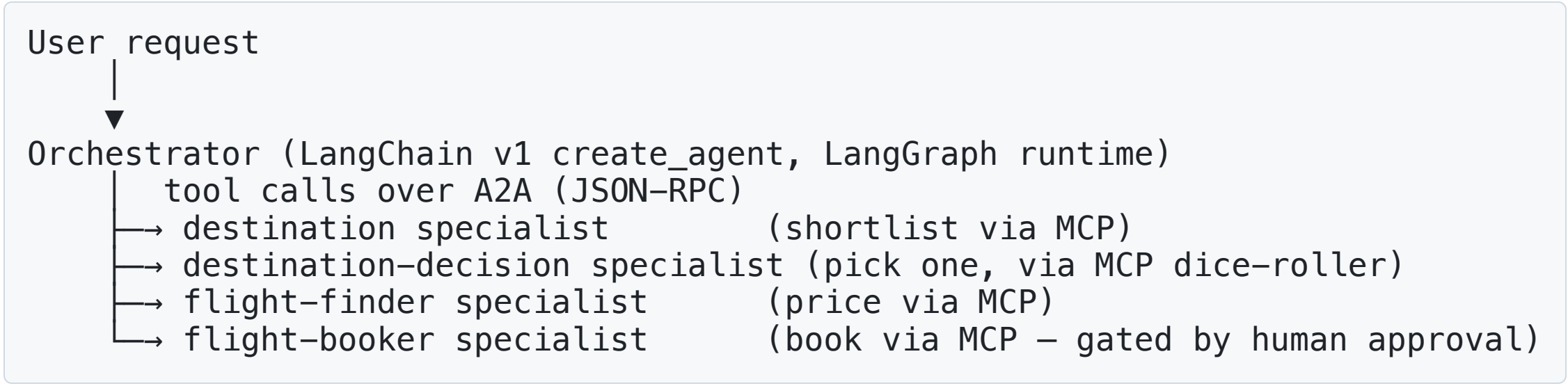
Multi-Agent Travel Booking System

The domain

A travel-booking assistant: given a free-text or structured trip request, it shortlists destinations, picks one, finds a flight, and books it — with a human approval gate before the booking actually happens.

Chosen because it has **natural multi-agent decomposition** (distinct specialist concerns) and a **clear point requiring human judgment** (committing money to a booking).

Architecture: orchestrator + 4 specialists



All inter-agent traffic routed through **agentgateway**; all specialist tool access via **MCP**.

Design choice: prompt-driven tool-calling, not a hand-authored graph

- The orchestrator is `create_agent` + 4 `@tool`-wrapped functions — **not** a bespoke `StateGraph` with manually wired edges
- Retry logic (try another destination if no flight found) lives in the system prompt, not in graph topology
- **Trade-off:** less explicit control flow to audit, but far less code to maintain, and the model's own reasoning handles edge cases a hand-authored graph would need explicit branches for

Design choice: MCP for every specialist tool call

- Each specialist is a thin FastAPI service using `MultiServerMCPClient` — no direct REST calls
- Benefits: uniform tool-discovery, routed through `agentgateway` for observability
- **Real trade-off discovered live:** MCP's streamable-HTTP session is stateful (session ID pinned to one backend instance) — scaling an MCP backend to multiple replicas behind a non-sticky load balancer broke it in production. Reverted; documented as a real limitation, not silently worked around.

Human-in-the-loop: the approval gate

- `book_flight` — the one irreversible, money-committing action — pauses via **LangGraph's** `interrupt()` before it actually books
- State persists through the checkpointer; a human resumes it later via `POST /approve`
- `GET /pending` lets a reviewer discover what's waiting, without already knowing a thread ID
- A small CLI tool (`approve_cli.py`) is the human-facing surface: polls, shows pending bookings, prompts Approve/Reject

Verified live: a real booking paused, was approved via the CLI, and completed with a real confirmation code. A separate request was rejected and genuinely left unbooked.

Trade-off, stated plainly: single-replica orchestrator

- The Postgres checkpointer makes compute horizontally scalable in principle — state lives externally, not in pod memory
- But there's no session affinity at the gateway: two replicas could race on the same `thread_id` 's checkpoint
- **Decision:** keep the orchestrator at `replicas: 1` until sticky routing or per-thread locking exists, documented directly in code and the GitOps manifest — not hidden

Evaluation: dataset design

13 examples across 5 categories, each with real `reference_outputs`:

Category	Tests
Happy path	Normal booking flow end-to-end
Missing values	Schema defaults apply correctly
No-flight	Every candidate exhausted, clean explanation
Retry	*(documented limitation — see next slide)*
Ambiguous	Free-text request interpretation

Evaluation: what we couldn't fake, and said so

No genuine "destination A fails, destination B succeeds" retry case exists in this system — the destinations/flights datasets have perfectly synced IATA codes, and the destination specialist pre-filters origin collisions before the shortlist is built.

Rather than mock the backend to force a fake retry (dishonest to the eval's purpose), this is **documented as a real architectural property worth a design discussion** — not hidden behind a passing test that doesn't test anything real.

Evaluation: 9 custom deterministic evaluators

Evaluator	Checks
`booking_succeeded` / `no_false_booking`	Booking happened iff it should have
`origin_correct` / `date_correct`	Booked flight matches the request
`requirements_correct`	Extraction matches expected fields
`tool_sequence_valid`	Tools called in the right order
`never_book_before_flight`	Hard invariant — can't be violated
`no_duplicate_destination_retry`	Retry logic doesn't repeat a failed pick
`retry_behavior_valid`	Retry happens exactly when expected

Plus one LLM-as-judge (`trip_matches_request`) for semantic fit — routed through the same LiteLLM gateway the agent itself uses, not a separate credential.

Real results — not simulated

Running the full dataset against the **live production deployment**:

- **12 / 13** on `booking_succeeded` and `trip_matches_request`
- The one failure: the LLM read "nothing too fancy on the budget" as `budget_level: low` — and the demo dataset has **zero** low-budget destinations. A genuine data-coverage gap, documented rather than patched around.
- **0 example-level errors** on the final clean run

Evaluation-driven development: 3 real bugs found and fixed

Running the eval suite against live infrastructure — not a mock — surfaced bugs nothing else had caught:

1. A **stale LangSmith dataset** silently masked every evaluator behind vacuous passes for weeks
2. `find_flight` could **leak a raw MCP error string** into its JSON contract instead of returning `null`
3. The extraction schema documented `"food"` as a valid **activity** when the backend only ever models it as a **vibe** — silently zeroing out real booking requests

All three fixed, redeployed, and re-verified live the same night.

Baseline-vs-candidate model experiment

Compared `openai-gpt` (gpt-4o-mini, baseline) against `gemini-pro` (gemini-2.5-pro, candidate) on the same live orchestrator:

- **openai-gpt:** ran the full dataset cleanly, no crashes, both concurrent and serial
- **gemini-pro:** crashed the orchestrator (`exit 137` , memory limit) under **both** concurrent and serial load — reproducible, not a fluke

Result: this isn't a quality comparison — it's an operational one. gemini-pro isn't viable in this deployment without raising memory limits and/or the liveness-probe tolerance. Reverted to the stable baseline.

Observability: two complementary signals

- **LangSmith** — agent/tool-call semantics, full distributed trace across all 5 services (A2A header propagation nests specialist spans under the orchestrator's run)
- **OTEL** — HTTP-layer traces/logs/metrics via OTLP, per-service
- **Verbose console logging** — live, human-readable trace of every tool call and result, printed to whichever terminal is running the orchestrator (built specifically for demo/ops visibility)

Business Value

Translating this into KPIs

- **Faster trip planning:** end-to-end booking (shortlist → pick → price → book) in seconds instead of a manual multi-step process
- **Controlled automation, not blind automation:** the HITL gate means the one financially consequential step always has a human checkpoint — full automation's speed, without full automation's risk
- **Fewer production incidents:** deterministic evaluators catch contract violations (wrong status, duplicate retries, booking-before-flight) before they reach a real customer
- **Faster root-cause time:** distributed tracing across 5 services means a bad outcome is traceable to the exact tool call that caused it, not a guessing game

Reliability as a feature, not an afterthought

Every claim in this deck is backed by a **live test against real infrastructure**, not a demo-only happy path:

- 3 real production bugs found and fixed via evaluation, same session
- A real infrastructure incident (MCP session-affinity) caught, reverted, and documented
- A real model-comparison experiment that found a real operational limit, not a clean win

This is the standard the platform should hold itself to going forward: **evaluation is not a one-time gate, it's how regressions get caught before customers do.**

Friction Log & Product Feedback

(Full detail: `docs/friction-log.md`)

What got in the way

1. `langchain-mcp-adapters` can turn a tool failure into a misleading "successful" string result instead of raising
2. **LangSmith's** `evaluate()` has no built-in way to reconcile a dataset against a changing definition — we had to build our own sync logic
3. `evaluate()` **'s per-example model** has no story for ordered or human-in-the-loop flows — both had to be hand-rolled outside it
4. **LangGraph's checkpointer** has no query for "which threads are currently paused on an interrupt" — essential for any real approval-inbox UI
5. `AsyncConnectionPool` **'s** `wait=False` **default** silently hides a dead Postgres connection until it takes the pod down under load

What worked well

- `interrupt()` + `Command(resume=...)` — genuinely pleasant once understood; worked correctly on the first live test
- `create_react_agent` → `create_agent` **migration** — clean, same runtime underneath, minimal changes needed
- **Cross-service trace propagation** — nested a 5-service distributed call chain into one LangSmith trace tree with very little code

Summary

- **Part 1:** conceptual self-hosted LangSmith architecture, grounded in real docs, with the storage-externalization trade-offs and scaling story InnovateCorp needs to evaluate it
- **Part 2:** a real multi-agent system, with a genuine human-in-the-loop gate, evaluated against live infrastructure — not simulated — that found and fixed 3 real bugs and one real operational limit in the process
- **Everything demoed tonight runs on real infrastructure right now**

Q&A